

Dependency flag soup needs some fiber

Ben Boeckel

<ben.boeckel@kitware.com>

version P2800R0, 2023-09-20

Table of Contents

[1. Abstract](#)

[2. Changes](#)

[2.1. R0 \(Initial\)](#)

[3. Introduction](#)

[4. Gathering Ingredients: Flag Soup Strategies](#)

[4.1. Simple flags](#)

[4.2. Autolinking](#)

[4.3. Compiler wrappers](#)

[4.4. Configuration tools](#)

[4.5. .pc files](#)

[5. Eating Your Greens: Usage Requirements](#)

[5.1. Terminology](#)

[5.2. Examples of Usage Requirements](#)

[5.3. Consistency Checking](#)

[5.4. Limitations](#)

[5.5. Representing Targets as Usage Requirements with Visibility](#)

[5.6. Application to Modules](#)

[5.6.1. Example](#)

[6. Dependency Recipes: Examples of Existing Projects](#)

[6.1. Sanitizers](#)

[6.2. Boost](#)

[6.3. MPI](#)

[6.4. Python](#)

[6.5. Qt](#)

[7. References](#)

Document number	ISO/IEC/JTC1/SC22/WG21/P2800R0
Date	2023-09-20
Reply-to	Ben Boeckel, ben.boeckel@kitware.com
Audience	EWG (Evolution), SG15 (Tooling)

1. Abstract

Dependency management and "how to use library X" has always been an important question to answer. Historical answers have included "assume that headers and libraries are found in the default locations" allowing simply using `-llibname` to use the compiled symbols, asking `libname-config` tools for flag information, compiler wrappers such as `mpicc`, or storing information in a `.pc` file and queried by `pkgconf` [[pkgconf](#)] or `pkg-config` [[pkg-config](#)]. There are many cases where they do not

compose, generate conflicts between each other, or introduce ambiguities. We need more structured information in order to accurately account for the variety of projects and how they are used in the wild. This is especially acute with modules where module clients may need to create separate BMI files from other clients needing the same module to satisfy implementation requirements.

2. Changes

2.1. R0 (Initial)

Description of why usage requirements and how flag soup is not good enough in the context of modules.

3. Introduction

Every project has some set of things that must be done in order to meaningfully consume it. This can include all kinds of things that tend to be communicated via compiler flags, linker flags, dynamic loader environment variables, plugin loading environment variables, and more. I refer to these things as "usage requirements" of a project. These can meaningfully change between libraries or other components of a project or even by what is consuming them (though in a limited way). Discovering these usage requirements is a core part of a build system's job. There are a number of mechanisms that have been used to gather usage requirements.

Few projects are islands that depend on no external code. While vendoring is common enough, it does not resolve the need to know how to meaningfully use a library. Much of the time, using a project is communicated via a set of flags to hand to a toolchain so that the in-source and runtime references to the library in question. How these flags are collected have been done in multiple ways over time. In any case, I call this method of gathering usage requirements "flag soup" as interpreting these flags is only truly possible by the toolchain itself in general. However, there is no isolation and one project specifying `-Lpatha -la` and another `-Lpathb -lb` can change meaning depending on the order they are provided to the toolchain if either `patha` or `pathb` contains the other library. Other more-specific flags may also be required to set up specific instances.

Instead, a more structured way of specifying "usage requirements" should be provided. This paper does not propose any specific way of doing so and instead argues for why such structured information is necessary. While CMake has "usage requirements", my vision goes much further in what kinds of information can and should be communicated through such a mechanism.

4. Gathering Ingredients: Flag Soup Strategies

Consuming dependencies is an important part of development. There are many ways to manage them today. Tools such as [\[Anaconda\]](#), [\[vcpkg\]](#), [\[conan\]](#), [\[spack\]](#), [\[Homebrew\]](#), [\[nix\]](#), [\[ports\]](#) (for FreeBSD as well as other platforms such as OpenBSD and Gentoo), [\[dpkg\]](#), [\[rpm\]](#), and more I'm sure I'm unaware of all exist to manage dependencies for C++ code. They all need to solve similar problems:

- knowing that project X requires project Y;
- making sure that Y is built in a way that is acceptable to X;
- telling X where Y is so that an acceptable version is used (especially if multiple installations of Y are supported).

X typically knows how to consume Y, but if this changes over time (say, Y needs a new project Z), X may need to know this to use newer versions of Y. Ideally, there would be a reliable way for Y to communicate this to X as needed.

These tools all solve these problems in different ways with different benefits and drawbacks. This paper is not focusing on the solutions these projects have come up with to stitch various projects together and instead is on making sure that projects can provide the required information in a rich enough way on their own. This is so that the problems can be solved once per project instead of once per project per distribution tool.

This section describes the various ways which have been used in various projects which involve what I call "flag soup". This is where consumers are handed flags (in some way) by their dependencies libraries to blindly pass to the toolchain so that the dependency can be used. Interpreting these flags is generally not possible and they also tend to assume a single command line dialect (typically GNU-like). While basic flags have compatible spellings across many compilers, there are differences in critical flags which may be important. Examples include:

- `-pthread` (GNU) versus `-Xcompiler -pthread` (CUDA)
- `-fPIC` (GNU) versus `-qpPIC` (IBM XL)
- `-fopenmp` (GNU) versus `-fiopenmp` (non-MSVC frontend of Intel oneAPI) versus `-openmp` (MSVC)

Any dependency which wishes to represent these kinds of things needs to be able to understand the context in which the dependency is being consumed. This means that it cannot just be a static lookup and there also needs to be some way to perform a dynamic lookup to understand the usage. In any case, there's no standard for conveying "I am using vendor X compiler" to these tools.

4.1. Simple flags

On "single installation prefix" (or "few installation prefix") systems such as Linux, BSD, or other Unix-like platforms where all dependencies tend to live in a few places, toolchains tend to search these places by default and simple flags such as `-llibname` to load the code (and the headers are already found implicitly).

In this case, the name of the libraries to load is all that is necessary and `-l` can be prefixed to the required library names and given to the toolchain to search for.

4.2. Autolinking

On Windows there is a mechanism to request a library be linked from the source code. This allows for dependencies to be used through just an include and library search path addition. The syntax for this is:

```
#pragma comment(lib, "libname")
```

During linking, this will cause it to act as if `libname.lib` were specified as an additional library to link to. However, the search path must be provided by other mechanisms.

4.3. Compiler wrappers

Some projects ended up writing their own "compiler" which wraps the base toolchain and adds the required flags as necessary while mediating between the consumer and the base toolchain.

The biggest issue with this approach is that it does not compose. These do not, generally, have a mechanism to stack them arbitrarily and instead expect to chain to the "lower" compiler by looking for it through the typical compiler names such as the `CC` environment variable or searching for "common" names for known compilers such as `cc`, `gcc`, or `clang`.

If the wrapper is a compiled tool, using it in a cross-compilation context can be difficult as it tends to be provided by the project itself. Even if not compiled, it may not understand how to incorporate flags such as `--sysroot` to re-root any paths it wants to pass to the underlying compiler.

4.4. Configuration tools

Projects such as `freetype` and `libpng` provide executables which provide the flags necessary to use the library. These can work well, but have real questions on how to support cross-compilation if provided in a compiled form (most tend to be shell scripts in practice).

These tools essentially manually implement `.pc` file resolution manually (and, in fact, are sometimes implemented in terms of `pkgconf` today). However, there is a lack of consistency at times, sometimes various features are missing, and there's no coordination with other ways of collecting dependency information.

4.5. .pc files

The `pkg-config` tool and its associated `.pc` files are widely used to provide these flags. These files are widely available and used on Unix-like platforms. Some projects provide them on Windows as well, but they are not widely available enough to be relied upon.

There is some affordance for getting dependencies and making sure that there is logic for the different linking requirements for static versus shared libraries, but the query language is a bit coarse and requesting the static version of library A while getting the shared version of library B is not easy to guarantee if both libraries have both variants available. There is also no way to provide different compilation flags between shared and static (e.g., to control the export macros on platforms such as Windows or AIX).

`.pc` files also tend to prefer `-L` and `-l` pairs which can confound each other. This seems to be done typically to support static and shared usage with a single set of flags.

5. Eating Your Greens: Usage Requirements

The term "usage requirements" comes from CMake's target-based model that was first released in CMake 2.8.12. Over time, more and more mechanisms have been added to this idea so that more than just include directories and libraries to link. Today, there are usage requirements for compile options, linker options, source files, compile definitions, and more. This does require some more work on the part of library authors to classify the flags they would like to use into "for me" and "for consumers". However, this is done at one central place.

CMake has a mechanism for the consuming library to influence these requirements as well. For example, it is possible to have a usage requirement of "if used by an executable, please also do X". CMake calls them generator expressions and uses its own syntax, but I'll avoid using its syntax here and instead use something that is (hopefully) clearer.

5.1. Terminology

I'll adopt some of CMake's terminology for the rest of this paper to discuss more concrete usage and limitations of the model.

- **library**: A logical unit of usage intended to be used by further targets. Libraries may be used at runtime by the dynamic loader (**shared** libraries) or a program directly (**module** libraries), by the linker as a group (**static** libraries), by the linker as a collection of object files (**object** libraries), or only by the build system (**interface** libraries).
- **executable**: A program that may be directly executed.
- **target**: A logical unit of usage. May be an executable, library, or just a collection of usage requirements.

5.2. Examples of Usage Requirements

CMake has a number of usage requirements today, but this is certainly not an exhaustive list of what makes sense. In fact, I feel that projects should be able to define their own. Actually using them requires some level of support in tools, but the ability to express the information is, I feel, a good starting place for representing the information in the first place.

- **compile definition**: a flag for the preprocessor (e.g., **-Dsome_flag=value**)
- **compile option**: a flag to give to the compiler (e.g., **-fstack-protector**)
- **compile feature**: a capability of the compiler that has unification logic with other features in the same "category" (e.g., **-std=c++17** and **-std=c++20** belong to the same "category")
- **precompile header**: headers to include in the precompiled header (of which most implementations only support one per TU); may also be used to indicate "use this one already-made PCH elsewhere too"
- **include directory**: paths to search for textual inclusion sources (e.g., collected and passed via **-I** or **-isystem** flags)
- **source**: sources that should be added to the consuming target's source list (e.g., **crt1.o**)
- **link option**: options to pass to the linker (possibly through another frontend) (e.g., **--as-needed**)
- **link directory**: directories to search for libraries specified by name (e.g., collected and passed via **-L** flags)
- **link library**: libraries to link to (e.g., **/usr/lib/libdl.so** or **-ldl**)

Further usage requirements I have wished for in the past:

- **rpath entry**: paths to add to rpath properties (to satisfy **@rpath/** library references on macOS)
- **runtime path**: paths to include in **PATH** when using a library (mainly for Windows)
- **runtime loader path**: like **runtime path**, but for the dynamic loader (**PATH** on Windows, **DYLD_LIBRARY_PATH** on macOS, and **LD_LIBRARY_PATH** on ELF platforms)
- **python path**: paths to collect and include in **PYTHONPATH** to use the library

Usage requirements can be used to represent all kinds of information. It is my feeling that projects should be able to declare and define their own usage requirements. Support for consuming particular usage requirements may differ as needed, though I would hope that compelling use cases become influential for supporting more usage requirements in more tools over time.

5.3. Consistency Checking

There may be checks for consistency across targets in a usage subgraph. For example, all code in a single program image must agree on the usage of the `-fPIC` flag when compiling its constituent objects.

5.4. Limitations

Usage requirements allow querying the consuming environment in order to modify the set of provided information. However, this is not an unlimited capability. Of note, nothing that can be queried may be modified through a usage requirement. If this were possible, statements akin to "this statement is false" are possible. More concretely, imagine a target (`eve`) that is able to set some property on the consuming target (e.g., "set property `use-alice` to `0` on the consuming target"). Another target `charlie` uses `alice` if it has and `use-alice` set to a non-`0` value. `alice` then has a usage requirement that consumers use `eve`. Now that the usage requirements of `eve` apply, it sets the `use-alice` property on `charlie` to `0`. This is now a paradox that cannot be resolved because `alice` should now **not** be used based on the property settings.

Therefore, usage requirements may only rely on properties of the context that they may not affect. This includes support for "intersectional usage requirements" such as "if X and Y are used, add usage requirement A".

5.5. Representing Targets as Usage Requirements with Visibility

When a layer of "visibility" is layered on top of usage requirements, they may also be used to represent the properties of the target for itself as well as for consumers. CMake gives these names:

- `interface`: usage requirements for consumers
- `private`: usage requirements for the target itself
- `public`: a combination of both `interface` and `private`

This additionally gives a classification of what flags are important for consuming modules.

5.6. Application to Modules

Modules have a narrower compatibility contract than the ABI of libraries and (see [\[P2581R2\]](#)). This means that there may be more than one BMI per build graph so that different consumers can import a module. This is because BMIs may involve internal data structures that are sensitive to flags such as `-std=` where the selection of the TU importing affects what BMIs may actually be loaded safely. There are many such flags that affect this compatibility in practice.

This narrow compatibility means that when consuming module interface units from an external project cannot rely on any BMI artifacts to be provided by that project and must be prepared to build BMIs for these interface units as part of its own build graph.

In terms of usage requirements, this manifests itself as variants of the target provided by the project for each unique usage pattern which gain some usage requirements from the direct consumer. An optimization may be made to share BMI builds if it is found that different importers can share BMIs while constructing the build graph.

In order to do this effectively, it is necessary to know what flags that end up being given to the compiler can be skipped when compiling these BMIs (the "safe" option is to give all flags and every potential importer gets its own BMI as necessary).

5.6.1. Example

As an example, say we have a project with two targets `use20` and `use23` which use the C++20 and C++23 standards, respectively. Both wish to import modules from a target named `has_modules`. While it is assumed that the ABI of the `has_modules` compile artifacts does not change between standards (the status quo with textual inclusion today), the BMI almost certainly cannot be reused and must be generated for each. This means that the build of the `use*` project must schedule BMI builds for `has_modules` with each `-std=` flag (and others as necessary).

If there is another `also_use20` target which further has flags to use another C dependency (in particular, `-I` flags for its headers), it would be nice to know that these flags are local to the `also_use20` target and that it can share BMI builds with `use20`. The classification of usage requirements into visibility classifications for targets gives build systems enough information to know what can be kept local to a given compilation and BMI requirement graph.

6. Dependency Recipes: Examples of Existing Projects

This section provides instances of projects where usage is not always straightforward and/or involve deeper knowledge of the toolchain itself. Each section has an example set of usage requirements in a Lisp-like syntax. They are not intended to be 100% accurate, but instead to give an idea of what kinds of things that might need to be constructed. Lisp is used because it is a syntax which can represent data structures with embedded code-like queries of things involving the consuming target.

In the examples, there is the `match` construction which queries the consuming environment. The particular queries here are meant for exposition and not meant to be a proposal of how usage requirements should be spelled, merely as a way to express the kinds of contextual queries that may be useful. The way that `match` is intended to work here is as:

```
(match {query}
  ({value1} {result})
  ({value2} {result})
  ({value3} {result}))
```

The `{query}` is an expression that represents a way to query the environment. This query is then matched against, in order, each `{valueN}` and the first value which matches results in the entire `(match)` expression with its corresponding `{result}`. A `_` value is a wildcard and matches any query result.

Queries used include:

- `'language`: The language of the consuming compilation (C, C++, CUDA, etc.).
- `'platform`: The platform of the consuming compilation (Windows, macOS, Linux, Android, etc.)
- `(property 'target-type)`: Query the target for its "type" (executable, shared library, interface library, etc.)
- `(property {some-other-symbol})`: Query the target for some other property and evaluate to its value.
- `(query {predicate})`: Pass the target to `{predicate}` to evaluate some higher-level information about it (e.g., "is a library" or "contains Fortran code").
- `(capture {name})`: The value may be captured and replaced in the `{result}` expression.

Additionally, there are some other patterns used which warrant further description:

- `(compile-definitions {name} ({with_value} {value}))`: Compile definitions can either be the name of a symbol or a pair with a name and value. These typically become `-Dname` and `-Dwith_value=value` flags.
- `(link-option (for-linker {value}))`: The flag needs to be passed directly to the linker. When constructing the command line, consider wrapping it as necessary to pass through any compiler frontends being preferred to using the linker directly (i.e., "add `-Wl`, if needed)

6.1. Sanitizers

Sanitizers are accessed via flags such as `-fsanitize=...`. These flags have both compiler and linker effects, namely generating the calls to extra checks in the code itself and adding the library providing the implementation of these extra checks to the linker.

In addition, for some sanitizers, if the code is loaded by an executable not using the sanitizer itself (e.g., a sanitized Python module loaded by a pre-existing interpreter), `LD_PRELOAD` is necessary to ensure that the sanitizer can install hooks early enough to provide their functionality.

```
(usage-requirements 'sanitizer-address
;; Give flags to the compiler and linker
(compile-option "-fsanitize=address")
(link-option "-fsanitize=address")
;; Ensure that the library is preloaded as needed
(runtime-preload "/path/to/libasan.so"))
```

6.2. Boost

While Boost is largely a header-only project, some of its libraries have additional compilation and linking requirements. For example, Boost.Archive can optionally use various compression libraries which must be provided if used. There is no mechanism for such optional dependencies when just providing flags a priori.

Boost also uses the autolinking mechanism to a note to the linker about libraries that are required based on the included headers. Projects which use Boost through more explicit means typically want to disable autolinking they prefer to provide unambiguous.

```
(usage-requirements 'boost-noautolink
;; Turn off "autolinking"
(compile-option "BOOST_ALL_NO_LIB"))
(usage-requirements 'boost-thread
(link-library "/path/to/libboost_thread.so")
;; Boost thread requires Boost's system library as well as its headers.
(target 'boost-system 'boost-includes))
(usage-requirements 'boost-system
(link-library "/path/to/libboost_system.so")
(target 'boost-includes))
(usage-requirements 'boost-includes
(include-directory "/path/to/includes/for/boost"))
```

6.3. MPI

MPI implementations have historically provided "compiler wrappers" named `mpicc` and `mpic++`. These are intended to be used instead of using the platform toolchain directly. These wrappers essentially just pass the required compiler and linker flags as necessary.

```
(usage-requirements 'mpi
  (include-directory "/path/to/includes/for/mpi")
  (link-library "/path/to/libmpi.so")
  ;; MPI libraries tend to assume that threads are available.
  (target 'threads)
  ;; Executables need to be launched using some tool to perform MPI's
  ;; functionalities. Mark as "mpi-" as there are details about using such
  ;; tools that need to be known anyways.
  (mpi-execution-launcher "/path/to/mpiexec"))
(usage-requirements 'threads
  (compile-option
    ;; Using threads is language-specific, so look at the *consuming* language
    ;; and select the correct flags.
    (match 'language
      ;; These flags go together and should be treated as a single unit.
      ('cuda ("-Xcompiler" "-pthread"))
      ;; All other languages should use `pthread`
      (_ "-pthread")))
    ;; The linker also needs to know (e.g., to add any additional libraries).
  (link-option
    (match 'language
      ('cuda ("-Xcompiler" "-pthread"))
      (_ "-pthread")))))
```

6.4. Python

Python is among one of the more complicated libraries to use due to the various ways in which it can be used and deployment strategies used for its distributions. Some consumers are creating modules, others are embedding the interpreter. Some module-using consumers additionally wish to limit their symbol usage using its "stable ABI" mechanism. These different consumption patterns are not easily expressible via a simple `.pc` file as the tools do not generally support a way to provide this kind of context (particularly for the "use the stable ABI as of version 3.x" case).

The expected usage pattern for Python is that the interpreter always provides the `libpython` ABI at runtime. In order to do this, libraries which expect to be used via an interpreter should *not* link to the `libpython` library. Modern linkers prefer to error if used symbols are not provided by a library given, so special flags (e.g., `-Wl, --unresolved-symbols=ignore-in-shared-libs`) are required to inform the toolchain to not care about missing symbols.

Things get more complicated when using these modules via further plugins. For example, a Python-using library may reasonably expect to be loaded by a Python interpreter (and so not link to its own `libpython`) while also wanting to be used from a plugin of another project that does not use Python itself. Here, the plugin must be told "you need to guarantee that `libpython` is provided" and so *it* must link to `libpython` even if it does not directly use any of its symbols itself. This can also require special flags (e.g., `-Wl, --no-as-needed`) to force the plugin to actually ensure that `libpython` is loaded.

```

(usage-requirements 'python-executable
  (executable "/path/to/python"))
(usage-requirements 'python-library
  (include-directory "/path/to/includes/for/python")
  (compile-definitions
    ;; Query the consumer to see if the stable ABI is requested.
    (match (property 'python-use-stable-abi)
      ;; The value of this property is the version of the stable ABI to use.
      ((capture 'version) ("Py_LIMITED_API" 'version))))
  ;; Not everything should link directly to `libpython`, so implement that
  ;; here.
  (link-library
    ;; Consuming executable should always link.
    (match (property 'target-type)
      ('executable "/path/to/libpython.so"))
    ;; If the consuming target is embedding or force linking, link to the
    ;; library.
    (match (property 'python-embed)
      (t "/path/to/libpython.so"))
    (match (property 'python-force-link)
      (t "/path/to/libpython.so")))
  (link-option
    ;; Consuming executable should always link.
    (match (property 'target-type)
      ('executable ()))
    ;; More conditionals here to select the correct flag for the given
    ;; platform and detecting other `python-` properties referenced above.
    (_ (for-linker "--no-as-needed")))))

```

6.5. Qt

On the whole, Qt is largely a normal dependency as far as direct usage is concerned. There are some extra tools Qt provides for various subsystems such as `moc`, `rcc`, and `uic`, but these are essentially additional toolchains which result in code generation that need to be used. Projects tend to handle these things explicitly and these use cases are not intended to be solved here.

Qt does get more complicated in two specific situations: use from executables on Windows and when using a static Qt. On Windows, Qt requires that its `qtmain` library is linked to, but only for executables in order to assist in creating a cross-platform `main` function. However, sometimes this is *not* wanted, so there should be a way to skip it (e.g., if some other framework is also assisting with `main` portability).

```

(usage-requirements 'qt-core
  (include-directory "/path/to/includes/for/qt")
  (compile-definitions
    ;; Allow targets to indicate that they would like to use `Q_` spellings for
    ;; all keywords.
    (match (property 'qt-portable-keywords)
      (t "QT_NO_KEYWORDS")))
  (link-library "/path/to/libQtCore.so")
  (target
    ;; Executables on Windows also need qt-main.
    (match (property 'target-type)
      ('executable

```

```
(match 'platform
  ('windows 'qt-winmain))))
;; Static libraries should also pull in plugins
(match (query is-static)
  ;; More platform and property checks here to select the right plugins as
  ;; necessary.
  (t ('qt-plugin-platform-xcb))))
(usage-requirements 'qt-winmain
  (link-library "/path/to/libQtWinMain.lib"))
```

7. References

- [Anaconda] Anaconda. <https://new.anaconda.com/>.
- [conan] conan. <https://conan.io/>.
- [dpkg] dpkg. <https://www.dpkg.org/>.
- [Homebrew] Homebrew. <https://brew.sh/>.
- [nix] nix. <https://nixos.org/>.
- [pkgconf] pkgconf. <http://pkgconf.org/>.
- [pkg-config] pkg-config. <https://www.freedesktop.org/wiki/Software/pkg-config/>.
- [P2581R2] Ruoso, Daniel. Specifying the Interoperability of Built Module Interface Files. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2581r2.pdf>.
- [ports] Ports (FreeBSD). <https://www.freebsd.org/ports/>.
- [rpm] rpm. <http://rpm.org/>.
- [spack] spack. <https://spack.io/>.
- [vcpkg] vcpkg. <https://vcpkg.io/>.